Part 2: Python Programming Basics

In this lesson, we learn basic Python programming concepts used in data processing.

Topics:

- Variables and data types
- Mathematical operations
- Type conversion
- String operations
- Lists
- Dictionaries

In [ ]:
```
name = "Himat"
age = 35
height = 1.75
is_student = False

print(name)
print(age)
print(height)
print(is_student)

print(type(name))
print(type(age))
print(type(height))
print(type(is_student))

hospital = "Hospital_A"
patient_id = 101
test_value = 5.6
test_completed = True

print(type(hospital))
print(type(patient_id))
```

```
print(type(test_value))
print(type(test_completed))
```

2. Mathematical Operations

Python supports basic mathematical operations.

These operations are useful for calculations and data analysis.

In [ ]:
```
a = 20
b = 6

print(a + b)   # Addition
print(a - b)   # Subtraction
print(a * b)   # Multiplication
print(a / b)   # Division
print(a // b)  # Floor division
print(a % b)   # Remainder
print(a ** b)  # Power
```

In [ ]:
```
price = 100
quantity = 5

total = price * quantity

print("Total Price:", total)
value1 = 10
value2 = 20
value3 = 30

average = (value1 + value2 + value3) / 3

print("Average:", average)
```

3. Type Conversion

Type conversion means changing data from one type to another.

Common functions:

- int() converts to integer
- float() converts to decimal number
- str() converts to string

Type conversion is important when cleaning datasets.

In [ ]:
```
age = "35"

print(type(age))

age = int(age)

print(age)
print(type(age))

price = "125.50"

price = float(price)

print(price)
print(type(price))
```

In [ ]:
```
patient_id = 1001

patient_id = str(patient_id)

print(patient_id)
print(type(patient_id))

value = "12.5"

# int(value) will produce an error

value = float(value)
```

```
print(value)
```

4. Working with Strings

Strings are used to store text data.

Python provides functions to clean and transform text.

Common operations:

- upper()
- lower()
- title()
- strip()
- replace()
- split()

In [ ]:
```
hospital = "Hospital A"

print(hospital.upper())
print(hospital.lower())
print(hospital.title())

hospital = "   Hospital A   "

print(hospital.strip())
```

In [ ]:
```
hospital = "Hospital_A"

print(hospital.replace("_", " "))

record = "Hospital_A,101,Glucose,5.6"

values = record.split(",")
```

```
print(values)
print(type(values))

text = "Data Processing with Python"

print(len(text))
```

5. Lists

A list stores multiple values in one variable.

Lists are useful for storing collections of data.

We can:

- Access elements
- Add new elements
- Remove elements
- Update values
- Sort values

In [ ]:
```
hospitals = ["Hospital_A", "Hospital_B", "Hospital_C"]

print(hospitals)
print(type(hospitals))

print(hospitals[0])
print(hospitals[1])
print(hospitals[-1])
```

In [ ]:
```
hospitals.append("Hospital_D")

print(hospitals)
```

In [ ]:
```
hospitals.remove("Hospital_B")
```

```
print(hospitals)

hospitals[0] = "Hospital_Main"

print(hospitals)
```

In [ ]:
```
test_values = [5.6, 7.2, 4.8, 6.1, 8.3]

print(test_values)

print(len(test_values))
print(min(test_values))
print(max(test_values))
print(sum(test_values))
```

In [ ]:
```
average = sum(test_values) / len(test_values)

print("Average test value:", average)
```

In [ ]:
```
test_values.sort()

print(test_values)
```

6. Dictionaries

A dictionary stores data using key-value pairs.

Dictionaries are useful for representing structured records.

For example, one hospital laboratory record can contain:

- Hospital name
- Patient ID
- Test name
- Test value

In [ ]:
```
patient = {
    "hospital": "Hospital_A",
    "patient_id": 101,
    "test_name": "Glucose",
    "test_value": 5.6
}

print(patient)
print(type(patient))
```

In [ ]:
```
print(patient["hospital"])
print(patient["patient_id"])
print(patient["test_value"])
```

In [ ]:
```
patient["test_value"] = 6.2

print(patient)
```

In [ ]:
```
patient["unit"] = "mmol/L"

print(patient)

print(patient.keys())
print(patient.values())
print(patient.items())
```

7. Combining Lists and Dictionaries

Real-world datasets contain multiple records.

We can store multiple dictionaries inside a list.

Each dictionary represents one record.

In [ ]:
```
patients = [
    {"patient_id": 101, "hospital": "Hospital_A", "test_value": 5.6},
    {"patient_id": 102, "hospital": "Hospital_B", "test_value": 7.2},
```

```
    {"patient_id": 103, "hospital": "Hospital_C", "test_value": 4.8}
]

print(patients)
```

In [ ]:
```
print(patients[0])
print(patients[1])
```

In [ ]:
```
print(patients[0]["hospital"])
print(patients[1]["test_value"])
```

8. Introduction to Pandas DataFrames

Pandas is a Python library used for data processing and analysis.

A DataFrame stores data in rows and columns.

We can convert Python dictionaries and lists into a DataFrame.

In [ ]:
```
import pandas as pd
df = pd.DataFrame(patients)

df
df.head()
df.shape
df.columns
df.dtypes
```

In [ ]:
```

```

In [ ]:
```
df["hospital"]
```

In [ ]:
```
df[["patient_id", "test_value"]]
```

In [ ]:
```
df["test_value"].mean()
df["test_value"].min()
```

```
df["test_value"].max()
```

9. Practice Exercise

Complete the following tasks:

1. Create variables for hospital name, patient ID, and test value.
2. Check the data type of each variable.
3. Convert a test value from string to float.
4. Create a list containing five test values.
5. Calculate the average test value.
6. Create a dictionary containing patient information.
7. Convert the dictionary into a Pandas DataFrame.

In [ ]:
```
hospital = "Hospital_A"
patient_id = 105
test_value = "7.8"
```

Task 1: Check data types

Task 2: Convert test_value to float

Task 3: Create a list of test values

Task 4: Calculate average

Task 5: Create a dictionary

Task 6: Convert dictionary into DataFrame